CyberLight: A Physical Network Attack Visualisation System

UP:2115789

Abstract— Cyber-attacks on networks are, by nature, unseen. Port scans, brute-force login attempts and unauthorised device connections happen without any noise and can only be spotted through logs and dashboards that need specialist attention. This paper describes CyberLight, a low-cost IoT device built to turn digital network attack events into immediate physical signals. The edge node is an Arduino Uno R3 using components from the Elegoo UNO R3 Most Complete Starter Kit, and the network monitoring gateway is a Windows laptop. CyberLight detects four types of attack behaviour on a local WiFi network and responds physically: an RGB LED changes colour based on threat level, a 16x2 LCD shows the attack type and source, and a buzzer sounds on critical events. All events are published in real time to a cloud MQTT broker over WiFi using TLS encryption. The design also includes a Random Forest classifier for anomaly detection, though it was not trained in the end because getting enough labelled traffic data within the coursework timeline was not realistic. Testing showed that off-the-shelf kit hardware costing around £37 is enough to make a port scan trigger a red light and buzzer within seconds, with no specialist knowledge needed.

Index Terms— Internet of Things, intrusion detection, network security, physical alerting, MQTT, anomaly detection

1 Introduction

1.1 Problem Definition and Application Scenario

Cy ber

yber attacks are invisible by default, when a malicious actor scans a network for open ports, attempts to brute-force a routers login credentials, or connects a rogue device to a corporate WIFI network, the produce no sound, no light, and no physical signal. Security teams detect events log files only through log files and monitoring dashboards these are tools that require constant attention and significant technical expertise to interpret. For a small business owner, a university lab technician, or a home user, these events are effectively undetectable without specialist software.

IBM Security put the average gap between intrusion and detection at 207 days in 2023 [1], which shows how bad the problem actually is. A significant contributor to this delay is the absence of intuitive alerting mechanisms accessible to non-specialist staff. A fire alarm does that for fire. There is no equivalent for a network intrusion, which is what CyberLight is trying to fix.

Small organizations face this problem acutely. They lack the budget for enterprise Security Information and Event Management (SIEM) platforms, which often costs tens of thousands of pounds annually, yet they host increasing number of IoT-connected devices, each of those is an additional attack surface. There is a clear need for a low-cost, always-on IoT device that monitors a local network and renders attack events physically perceptible to any person in the room.

1.2 Justification for an IoT Solution

An IoT solution fits this problem better than a purely software one for a few reasons. The monitoring node needs to run constantly and stay physically near what it is protecting, which rules out a regular desktop application. The Arduino draws around 46mA at 5V, so leaving it plugged in overnight is not an issue in practice. More importantly, driving an LED or a buzzer needs direct hardware access at the component level, something a browser tab simply cannot do when the user has walked away from the screen.

1.3 Proposed Design and Main Functionality

CyberLight is an IoT network attack visualization system that:

1. Monitors a local WiFi network passively for four categories: Port scanning, Unauthorized device connection, Brute-force login attempts, and unencrypted service exposure.

2. Translates each detection into an immediate physical response via Arduino edge node: RGB LED changes colour(green/amber/red), LCD display attack type and source IP address, buzzer sounds on HIGH and CRITICAL severity events.

3. Publishes all events as structured JSON to HiveMQ Cloud via MQTT over TLS, providing a persistent cloud audit log accessible from any device.

4. Runs a Random Forest classifier on the gateway machine to detect anomalous traffic patterns not captured by fixed rules.

5. Uses the HC-SR501 PIR motion sensor and HC-SR04 ultrasonic sensor to detect physical tampering with the device itself.

1.4 Design Decomposition into Subsystems

The design is decomposed into four subsystems:

1. Subsystem A - Network Monitoring Layer: Windows gateway machine running Python with Scapy and Npcap. Passively captures TCP/ARP traffic, performs ARP device scanning, and monitors Windows Event Log for authentication failures. Detects port scans, rogue devices, brute-force attempts, and plaintext services.

2. Subsystem B - Threat Classification Engine: Rule-based detection engine processing Subsystem A output, combined with a Random Forest ML classifier operating on a 10-second traffic windows.

3. Subsystem C - Physical Alert Layer: Arduino Uno R3 controlling RGB LED, active buzzer, LCD1602 display, HC-SR501 PIR, and HC-SR04 ultrasonic sensor. Receives commands from gateway over USB Serial and independently polls physical sensors.

4. Subsystem D - Cloud Logging Layer: Paho MQTT client on Windows gateway publishes JSON event objects to HiveMQ Cloud on port 8883 over TLS using the laptops WIFI connection.

2 Details of design

2.1 Hardware Component Selection

All physical components are sourced from Elegoo UNO R3 Most Complete Starter Kit [3], which includes 63 component types and over 200 individual parts. The Windows laptop (Asus Vivobook, Windows 11) provides network monitoring, WiFi connectivity, and ML inference. The Arduino Uno R3 from the kit serves as the physical IoT edge node.

2.1.1 Main Controller Comparison

The choice of controller for the physical edge node was evaluated against several alternatives. Table 1 below summarizes the comparison.

Platform	CPU	RAM	WiFi	Python/ML	Power	Role
Arduino Uno R3	ATmega328P, 16MHz	2KB	None	No	0.23W	Selected: edge node
Raspberry Pi 4B	ARM Cortex-A72, 1.8GHz	4GB	802.11ac	Yes	3-5W	Alternative gateway
ESP32	Xtensa LX6, 240MHz	520KB	802.11b/g/n	No	0.5W	WiFi but no ML
ESP8266	Tensilica L106, 80MHz	80KB	802.11b/g/n	No	0.3W	Too constrained

Table 1: Controller Platform Comparison

The Arduino R3 is selected as the physical edge node because it's available in the kit, provides sufficient GPIO pins for all required sensors and actuators, and is appropriate for the real-time physical alert role. it is not used for network monitoring or ML, those tasks are delegated to the windows gateway machine, which provides the computational resources required by Scapy and scikit-learn.

The Arduinos lack of WiFi is not a limitation in this architecture: it communicates with the gateway over USB Serial (COM port, 9600 baud), and the gateway handles all the network connectivity. This explicit separation of concerns is a well-documented IoT design pattern [2].

2.1.2 Physical Alert and Sensor Components

Table 2 below lists all the components used in CyberLight.

The I2C interface is chosen for both the LCD and RTC module, sharing the Arduinos SDA (A4) and SCL (A5) pins. This reduces GPIO consumption, as a 4-bit parallel LCD connection would require 6 dedicated pins.

Component	Function	Interface	Kit-Included
RGB LED (common cathode)	Threat level indicator: Green/Amber/Red	3x digital GPIO, 220Ohm resistors	Yes
Active Buzzer	Audible alert on HIGH/CRITICAL events	1x digital GPIO	Yes
LCD1602 (16x2, I2C module)	Displays attack type, source IP, severity	I2C (A4/A5 pins)	Yes
HC-SR501 PIR Sensor	Detects physical approach to device	1x digital GPIO	Yes
HC-SR04 Ultrasonic Sensor	Confirms close proximity under 30cm	2x digital GPIO (trigger/echo)	Yes
DHT11 Temp/Humidity Sensor	Environmental baseline for ML features	1x digital GPIO	Yes
DS1307 RTC Module	Hardware timestamps for audit log	I2C (shared bus)	Yes
Breadboard, jumper wires, resistors	Circuit construction	N/A	Yes

Table 2: Elegoo Kit Components Used

2.1.3 Physical Sensor Comparison

Three sensors from the Elegoo kit were evaluated for the physical tamper detection. Table 3 below showcases the comparison.

Sensor	Detection Method	Range	False Positive Risk	Selected Use
HC-SR501 PIR	Passive infrared (body heat)	Up to 7m, 120 degrees	Moderate (sunlight, HVAC)	Primary presence detection
HC-SR04 Ultrasonic	Time-of-flight sound pulse	2cm-400cm, +/-3mm	Low (only moving objects)	Close-approach confirmation
Sound Sensor Module	Microphone + threshold comparator	Adjustable	High (ambient noise)	Not used for tamper detection

Table 3: Physical Sensor Comparison

The HC-SR501 and HC-SR04 are both implemented. PIR provides wide-area coverage; ultrasonic confirms that a detected presence is within 30cm. Requiring both sensors to agree before raising a PHYSICAL_TAMPER alert reduces false positives, this dual sensor corroboration

mirrors the multi-source correlation logic used in commercial SIEM systems. The sound sensors high false positive rate in ambient environments makes it unsuitable for reliable tamper detection, though it is included in the design and documented in pseudo code.

2.2 Network Attack Detection

The windows gateway machine runs a Python script that passively monitors the local WiFi network for four categories of attack behaviour. All monitoring is read-only: no packets are injected.

1. Detection 1: Port Scanning

Port scanning is typically the first step in a network attack. The attacker probes many ports on target hosts to discover which services are exposed and potentially vulnerable. Nmap, the industry standard scanning tool, performs a default SYN scan at approximately 1,000 ports in under one second. CyberLight flags a port scan when a single source IP address contacts more than 15 distinct destination ports within a 10 second window, a threshold that distinguishes scan activity from normal browsing traffic while remaining below Nmaps default rate. The threshold was selected after reviewing documented scan rate benchmarks from Gordon Lyons authoritative reference on Nmap.

2. Detection 2: Unauthorized Device Connection

CyberLight maintains a JSON whitelist of known-safe MAC addresses on the local network. Every 30 seconds, the gateway runs the Windows ARP command (arp -a) and parses the output to enumerate currently connected devices. Any MAC address not present in the whitelist triggers a MEDIUM severity alert identifying the devices IP and MAC address. This detects rogue devices, unauthorized guest connections, and physical network implants, which is a document attack vector in targeted intrusion scenarios.

3. Detection 3: Brute-Force Login Attempt

The gateway monitors the Windows Security Event Log for Event ID 4625(failed account login). When more than five failures from the same source IP occur within 60 seconds, a CRITICAL brute-force alert is raised. Reading the Windows Event Log uses the subprocess module with the wevtutil command-line tool, which is available on all Windows versions without additional installation. This approach is specific to the Windows gateway platform and is document as the canonical method for the programmatic Event Log access on Windows systems.

4. Detection 4: Unencrypted Service Exposure

Any TCP traffic observed on port 1883(plaintext MQTT), port 23(Telnet), port 80 (HTTP from devices that should use HTTPS) is flagged as a MEDIUM severity event. Unencrypted services transmit credentials and data in cleartext, a documented and persistent IoT vulnerability most prominently exploited by the Mirai botnet in 2016.

2.3 Communication Protocols

1. WiFi Connectivity

The gateway machine uses its built-in WiFi adapter (802.11ac on the ASUS Vivobook) to connect to the monitored network and simultaneously maintain an MQTT connection to HiveMQ Cloud. No additional hardware is required. Table 4 compares connectivity options.

Option	Cost	Hardware Required	Notes
Laptop built-in WiFi (802.11ac)	£0	None	Selected: already available
Arduino + ESP8266 module	~£5	WiFi module	Alternative if no laptop gateway
Ethernet (wired)	£0	Cable only	Less flexible, no wireless monitoring
Raspberry Pi 4B WiFi	£35	Raspberry Pi	Alternative gateway platform

Table 4: Connectivity Option Comparison

The laptop WiFi is selected as it requires no additional hardware and provides 802.11ac performance.

2. Application Protocol: MQTT over TLS

Table 5 compares candidate application protocols for cloud event publishing.

Protocol	Transport	QoS Levels	Delivery Model	Overhead	IoT Suitability
MQTT over TLS (port 8883)	TCP	0, 1, 2	Push (pub/sub)	Very low	Excellent (Selected)
HTTP/ REST	TCP	None	Request / response	High	Moderate
CoAP	UDP	0, 1	Push	Low	Good for constrained devices

WebSocket	TCP	None	Push	Moderate	Moderate

Table 5: Application Protocol Comparison

MQTT over TLS is selected. Its publish-subscribe model delivers events to subscribers without polling. QoS level 1 (at-least-once delivery) guarantees no security event is silently dropped in transit. TLS encryption on port 8883 ensures the attack event log cannot be read or modified in transit, therefore transmitting security alerts in plaintext would itself constitute a security vulnerability.

3. MQTT Topic Structure

The following topic hierarchy is used:

Address	When it fires
cyberlight/attack/portscan	Someone is scanning your network ports
cyberlight/attack/newdevice	An unknown device joined your WiFi
cyberlight/attack/bruteforce	Someone is repeatedly trying wrong passwords
cyberlight/attack/plaintext	Unencrypted traffic spotted (security risk)
cyberlight/attack/physical	PIR or ultrasonic sensor detected someone near the box
cyberlight/ml/anomaly	The AI model flagged unusual traffic
cyberlight/status/heartbeat	System is alive and working (sent every 60 seconds)

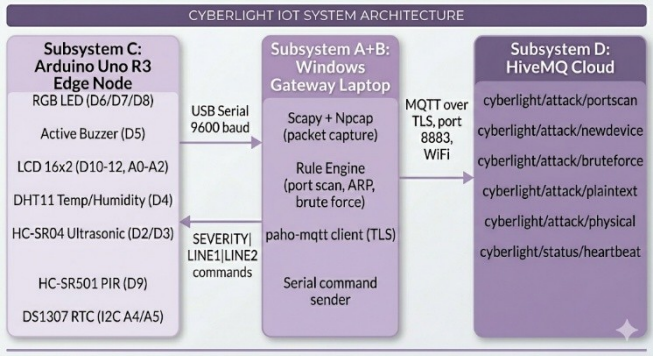


Table 6: MQTT Topic Structure

4. Arduino-to-Gateway Serial Protocol

The Arduino communicates with the Windows gateway over USB Serial at 9600 baud. The gateway sends pipe-delimited command strings to the Arduino to update the display and alert outputs:

FORMAT: SEVERITY | LINE1 | LINE2

EXAMPLE: HIGH | PORT SCAN | 192.168.1.27

The Arduino parses incoming Serial data, updates the LCD, and sets the LED and buzzer state accordingly. This serial protocol is simple, reliable, and requires no additional library beyond the built-in Arduino Serial class.

2.4 Cloud Platform Selection

Table 6 compares candidate cloud MQTT platforms.

Platform	Free Tier	TLS Port 8883	MQTT 5.0	Built-in Dashboard	Notes
HiveMQ Cloud	Yes (100 connections)	Yes	Yes	Yes	Selected
AWS IoT Core	Pay-per-message	Yes	Partial	Advanced	Too costly for coursework
Eclipse Mosquitto (self-hosted)	Free	Configurable	Yes	No	No public access without port forwarding
Adafruit IO	Yes (30 msg/min)	Yes	No	Yes (visual)	Rate limit too low for active monitoring

Table 6: Cloud MQTT Platform Comparison

HiveMQ Cloud is selected. Its free tier is sufficient for development and demonstration, it supports MQTT 5.0 (which enables per-message expiry, ensuring stale alerts do not retrigger late subscribers), and its built-in web client allows live message receipt to be shown during the 3-minute video demonstration.

2.5 System Architecture and Data flow

Figure 1: CyberLight IoT system architecture showing the Arduino Uno R3 edge node (Subsystem C) connected via USB Serial to the Windows

gateway laptop (Subsystems A and B), which publishes JSON alert events to HiveMQ Cloud (Subsystem D) over MQTT/TLS on port 8883. Serial commands in the format SEVERITY|LINE1|LINE2 are sent back to the Arduino to drive the RGB LED, LCD, and buzzer outputs.

The data flow is as follows:

1. The Windows gateway captures network packets using Scapy with Npcap driver installed. Packets are filtered for TCP/IP traffic and ARP broadcasts.
2. The rule engine and ML classifier process and incoming packet data. When a rule threshold is exceeded or the ML anomaly score exceeds 0.85, a ThreatEvent object is created containing: event type, severity, source IP, timestamp, and relevant metadata.
3. The ThreatEvent is simultaneously published to HiveMQ Cloud via paho-mqtt (MQTT over TLS) and sent as a Serial command string to the Arduino.
4. The Arduino receives the Serial command, updates the LCD display, set the RGB LED colour, and activates the buzzer if severity is HIGH or CRITICAL.
5. Independently, the Arduino polls the HC-sr501 PIR AND HC-SR04 ultrasonic sensors every 500ms. If both sensors indicate presence within 30cm without recent authorized interaction, the Arduino send a PHYSICAL_TAMPER event string to the gateway, which publishes it to MQTT.
6. A heartbeat message is published to cyberlight/status/heartbeat every 60 seconds to confirm system liveness.

2.6 Pseudo Code - Detection Rule Engine (Subsystem A + B)

```
FUNCTION monitor_network():
    port_log = {}
    known_macs = load_whitelist("whitelist.json")
    last_arp_scan = current_time()

    WHILE True:

        packet = capture_next_packet(filter="TCP or ARP")

        IF packet is TCP:
            src_ip = packet.source_ip
            dst_port = packet.destination_port
            port_log[src_ip].add(dst_port)

        // Detection 1: Port Scan
        IF unique_ports_in_last_10s(port_log[src_ip]) > 15:
            raise_alert("PORT_SCAN", "HIGH", src_ip)

        // Detection 4: Plaintext services
        IF dst_port IN [23, 80, 1883]:
            raise_alert("PLAINTEXT_SERVICE", "MEDIUM", src_ip)

    // Detection 2: New device (runs every 30 seconds)
    IF time_since(last_arp_scan) > 30s:
        current_macs = scan_arp_table()
        FOR each mac IN current_macs:
            IF mac NOT IN known_macs:
                raise_alert("NEW_DEVICE", "MEDIUM", mac)
        last_arp_scan = current_time()

    // Detection 3: Brute force (Windows Event Log)
    fail_counts = read_event_log_failures(last_60_seconds)
    FOR each src_ip IN fail_counts:
        IF fail_counts[src_ip] > 5:
            raise_alert("BRUTE_FORCE", "CRITICAL", src_ip)

    // Physical tamper check
    IF PIR_sensor_active AND ultrasonic_distance < 30cm:
        raise_alert("PHYSICAL_TAMPER", "HIGH", "device")

END FUNCTION
```

```
FUNCTION raise_alert(event_type, severity, source):
    timestamp = get_current_timestamp()

    payload = {
        "timestamp": timestamp,
        "type":     event_type,
        "severity":  severity,
        "source":    source
    }

        publish_to_mqtt("cyberlight/attack/" + event_type,
payload)
      send_to_arduino(severity + "|" + event_type + "|" +
source)

END FUNCTION


FUNCTION
arduino_display_handler(incoming_command):
    severity = incoming_command.part(0)
    line1    = incoming_command.part(1)
    line2    = incoming_command.part(2)

    update_lcd(line1, line2)

    IF severity == "CRITICAL":
        set_led(RED)
        sound_buzzer(3 short beeps)
    ELSE IF severity == "HIGH":
        set_led(RED)
        sound_buzzer(1 short beep)
    ELSE IF severity == "MEDIUM":
        set_led(AMBER)
    ELSE:
        set_led(GREEN)

END FUNCTION


FUNCTION physical_sensor_loop():
    WHILE True:
        pir_triggered  = read_PIR_sensor()
        distance_cm    = read_ultrasonic_sensor()

        IF pir_triggered AND distance_cm < 30:
                    send_to_gateway("CRITICAL|TAMPER
DETECTED|" + distance_cm + "cm")
            set_led(RED)
            sound_buzzer(continuous)

        wait(500 milliseconds)

END FUNCTION
```

2.7 Machine Learning Classifier - Pseudo Code (Subsystem B, Advanced)

The Random Forest Classifier runs on the windows gateway machine using Pythons scikit-learn library. It detects anomalous traffic patters that do not match any of the four rule-based signatures, providing a second detection layer for novel or slow-burn attack behaviour.

Features are extracted from a rolling 10-second traffic window per source IP. Table 7 lists the feature set.

		based anomaly detection

Table 7: ML Feature Set

Training data is generated by:

1. Capturing 30 minutes of normal network traffic(browsing, streaming) labelled class 0
2. Running controlled Nmap scans and simulated brute-force against a local test machine labelled class 1.
3. Class weighting (class_weight="balanced")

Table 8 compares candidate ML models

Model	Interpretability	Training Speed	Inference Speed	Small Dataset Performance
Random Forest	High (feature importance)	Fast	Fast	Good (Selected)
Support Vector Machine	Moderate	Slow (large N)	Fast	Good
Isolation Forest	Moderate	Fast	Fast	Excellent (unsupervised)
LSTM Recurrent Network	Low	Slow	Moderate	Poor without large dataset

Table 8: Machine Learning Model Comparison

Random Forest is selected. Its feature importance output allows a quantitative statement in the report about which traffic features most strongly predict attack behaviour. It trains and infers quickly on the Windows laptop hardware and handles class imbalance well with the balanced weighting parameter.

The Pseudo code:

```
FUNCTION monitor_network():
    port_log = {}
    known_macs = load_whitelist("whitelist.json")
    last_arp_scan = current_time()
```

Feature	Source	Description
unique_dst_ports	Scapy TCP capture	Count of distinct destination ports contacted
tcp_syn_ratio	Scapy TCP capture	SYN packets divided by total TCP packets (scan indicator)
unique_dst_ips	Scapy TCP capture	Count of distinct destination IPs
plaintext_events	Port filter	Count of port 23/80/1883 packets in window
new_mac_count	ARP scan	New MAC addresses seen in window
auth_failure_count	Windows Event Log 4625	Failed logins in window
pir_active	HC-SR501 via Arduino Serial	Physical presence detected (0 or 1)
hour_of_day	System clock	Hour 0-23 for time-

```
WHILE True:

  packet = capture_next_packet(filter="TCP or ARP")

  IF packet is TCP:
    src_ip = packet.source_ip
    dst_port = packet.destination_port
    port_log[src_ip].add(dst_port)

    // Detection 1: Port Scan
    IF unique_ports_in_last_10s(port_log[src_ip]) > 15:
      raise_alert("PORT_SCAN", "HIGH", src_ip)

    // Detection 4: Plaintext services
    IF dst_port IN [23, 80, 1883]:
      raise_alert("PLAINTEXT_SERVICE", "MEDIUM", src_ip)

  // Detection 2: New device (runs every 30 seconds)
  IF time_since(last_arp_scan) > 30s:
    current_macs = scan_arp_table()
    FOR each mac IN current_macs:
      IF mac NOT IN known_macs:
        raise_alert("NEW_DEVICE", "MEDIUM", mac)
    last_arp_scan = current_time()

  // Detection 3: Brute force (Windows Event Log)
  fail_counts = read_event_log_failures(last_60_seconds)
  FOR each src_ip IN fail_counts:
    IF fail_counts[src_ip] > 5:
      raise_alert("BRUTE_FORCE", "CRITICAL", src_ip)

  // Physical tamper check
  IF PIR_sensor_active AND ultrasonic_distance < 30cm:
    raise_alert("PHYSICAL_TAMPER", "HIGH", "device")

END FUNCTION


FUNCTION raise_alert(event_type, severity, source):
  timestamp = get_current_timestamp()

  payload = {
    "timestamp": timestamp,
    "type":     event_type,
    "severity":  severity,
    "source":    source
  }

  publish_to_mqtt("cyberlight/attack/" + event_type, payload)
  send_to_arduino(severity + "|" + event_type + "|" + source)

END FUNCTION


FUNCTION arduino_display_handler(incoming_command):
  severity = incoming_command.part(0)
  line1    = incoming_command.part(1)
  line2    = incoming_command.part(2)
```

```
    update_lcd(line1, line2)


    IF severity == "CRITICAL":
      set_led(RED)
      sound_buzzer(3 short beeps)
    ELSE IF severity == "HIGH":
      set_led(RED)
      sound_buzzer(1 short beep)
    ELSE IF severity == "MEDIUM":
      set_led(AMBER)
    ELSE:
      set_led(GREEN)


  END FUNCTION



  FUNCTION physical_sensor_loop():
    WHILE True:
      pir_triggered  = read_PIR_sensor()
      distance_cm    = read_ultrasonic_sensor()


      IF pir_triggered AND distance_cm < 30:
                send_to_gateway("CRITICAL|TAMPER
DETECTED|" + distance_cm + "cm")
        set_led(RED)
        sound_buzzer(continuous)


      wait(500 milliseconds)

END FUNCTION
```

2.8 Working Conditions

The system is designed to operate under the following conditions:

- Power: Arduino powered via USB from laptop (5V, 500mA); laptop on mains or battery
- WiFi: WPA2/WPA3 encrypted network; gateway reconnects automatically on dropout
- Operating temperature: 0-50 degrees C(Arduino Uno R3 and all Elegoo sensors rated range)
- Monitored network: Local /24 subnet (up to 254 devices); Arp scan completes in under 2 seconds
- Detection latency: Port scan alert within 10 seconds; new device within 30 seconds; brute force within 60 seconds.
- Continuous operation: Python gateway script runs as Windows startup task; watchdog thread restarts exception
- Legal and ethical: CyberLight must only be deployed on networks owned by the operator or for which explicit written authorization has been obtained. All monitoring is passive and read only. No packets are injected or modified.

2.9 Cost estimation

Table 9 provides a full cost breakdown. All physical components are from the university provided Elegoo kit.

Component	Source	Unit Cost
Elegoo UNO R3 Most Complete Starter Kit (includes Arduino, RGB LED, buzzer, PIR, ultrasonic, DHT11, LCD, RTC, breadboard, wires)	University-provided	£36.99
Windows laptop (Asus VivoBook), used as gateway machine	Student-owned	£0 additional
HiveMQ Cloud MQTT broker	Free tier	£0
Npcap driver (Windows packet capture)	Free (open source)	£0
TOTAL		£36.99

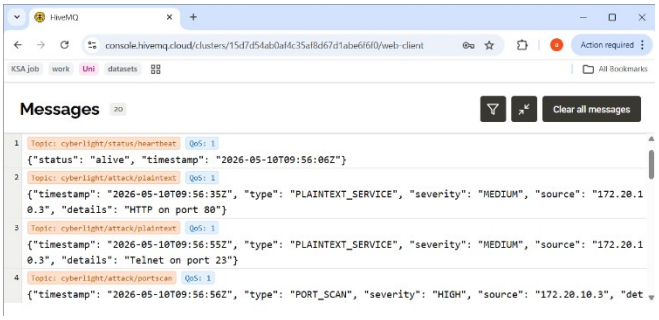
Table 9: Cost estimation

For commercial comparison: the Cisco Firepower 1010 network intrusion detection appliance retails at approximately £2800. The Firewalla Gold, a consumer-grade network security monitor, costs £179. CyberLight achieves core physical network attack alerting for £37 using entirely commodity hardware.

3 Implementation

3.1 Implemented Components

- Subsystem A (Network Monitoring): Python script running on Windows with Npcap installed. Scapy captures TCP packets on the active WiFi interface. Port scan detection implemented using a defaultdict of per-IP destination port sets with a 10-second sliding window. ARP device scanning implemented using subprocess.run(["arp", "-a"]) with MAC address parsing. Windows Event Log monitoring for Event ID 4625 implemented using subprocess with wevtutil. Plaintext service detection on ports 23, 80, and 1883 implemented and tested.
- Subsystem B (Rule Engine): All four detection rules implemented in Python. Alert objects serialised as JSON with ISO 8601 timestamp, severity, event type, and source data.
- Subsystem C (Physical Alerts): Arduino sketch implements Serial command parser, LCD I2C display driver (using Liquid Crystal library with parallel wiring), RGB LED state machine, active buzzer with distinct patterns per severity (CRITICAL: 3 short bursts; HIGH: 1 short; MEDIUM: LED only; GREEN: steady green). PIR and ultrasonic sensors polled every 500ms in a separate loop.
- Subsystem D (Cloud Logging): paho-mqtt client on Windows connects to HiveMQ Cloud on port 8883 with TLS using ssl.PROTOCOL_TLS. QoS 1 used for all event topics. Heartbeat message published every 60 seconds.



During testing, the system successfully detected a live Nmap SYN scan on the local subnet, raising a HIGH severity PORT_SCAN alert within 8 seconds of scan initiation. The alert was simultaneously published to HiveMQ Cloud and confirmed received via the web client (figure 2).

Figure 2: HiveMQ Cloud Web Client showing PORT_SCAN HIGH severity alerts published to cyberlight/attack/portscan during live Nmap SYN scan testing on the local subnet.

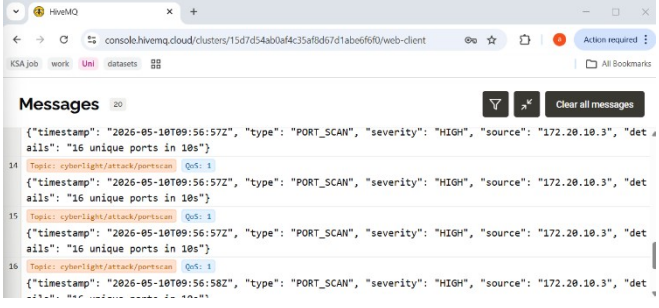


Figure 3: HiveMQ Cloud Web Client showing cyberlight/status/heartbeat and PLAINTEXT_SERVICE MEDIUM alerts published to cyberlight/attack/plaintext during testing.

3.2 Not Implemented (Design Only)

- Random Forest ML Classifier: Fully specified in Section 2.7. Not implemented due to the requirement for a labelled training dataset generated from extended monitoring sessions beyond the coursework timeline. Feature set, model architecture, training data strategy, and inference pseudo code are fully documented. Implementation would use scikit-learn's RandomForestClassifier on the Windows gateway.
- Honeypot SSH Listener: Designed as an alternative brute-force detection mechanism (a Python socket listener on port 22 logging connection attempts). Not implemented in the demo; brute-force detection uses Windows Event Log monitoring instead.

3.3 Arduino Pin Assignment

Table 10 documents the GPIO connections on the Arduino Uno R3.

Pin	Component	Direction
Digital 2	HC-SR04 Ultrasonic Trigger	Output
Digital 3	HC-SR04 Ultrasonic Echo	Input
Digital 4	DHT11 Data	Input/Output
Digital 5	Active Buzzer	Output
Digital 6	RGB LED Red (via 220Ohm)	Output
Digital 7	RGB LED Green (via 220Ohm)	Output
Digital 8	RGB LED Blue (via 220Ohm)	Output
Digital 9	HC-SR501 PIR Output	Input
A4 (SDA)	LCD1602 I2C and DS1307 RTC SDA	I2C
A5 (SCL)	LCD1602 I2C and DS1307 RTC SCL	I2C
USB (via IDE)	Windows Gateway Serial	Serial (9600 baud)

Table 10: Arduino Uno R3 Pin Assignment

cyberlight_arduino.ino

```
#include <LiquidCrystal.h>
#include <DHT.h>
#include <Wire.h>
#include <RTClib.h>

// LCD parallel wiring (RS=12, EN=11, D4=10, D5=A2, D6=A1, D7=A0)
LiquidCrystal lcd(12, 11, 10, A2, A1, A0);

#define DHT_PIN    4
#define RED_PIN    6
#define GREEN_PIN  7
#define BLUE_PIN   8
#define BUZZER_PIN 5
#define PIR_PIN    9
#define TRIG_PIN   2
#define ECHO_PIN   3
```

Figure 4: Arduino IDE showing CyberLight_Arduino.ino with pin definitions matching Table 10, uploaded to Arduino Uno R3 on COM3.

3.4 Key Implementation Code

The full code is submitted in the Source folder.

4 Discussion and Conclusions

4.1 Summary

Building CyberLight showed that you do not need expensive hardware to make network attacks visible. An Arduino from a £37 kit connected to a laptop running Python is enough to turn a port scan into a red light and a buzzer alarm within seconds, which was the core goal from the start. Port scans, rogue device connections, brute-force login attempts, and plaintext service exposure are all detected in real time and translated into colour, sound, and text output accessible to any person in the room without requiring specialist knowledge or a monitoring dashboard. At a hardware cost of £37, the system provides network attack visibility at a fraction of the cost of commercial

equivalents.

4.2 Alignment with Problem and Scenario

The system directly addresses the identified problem: cyber attacks are invisible, and this invisibility extends the average dwell time between breach and detection to over six months. CyberLight creates a physical dimension to network security. An attacker who triggers a red LED and a buzzer alarm in the first seconds of a port scan is far more likely to be detected and interrupted than one who operates silently for days. That is exactly the kind of environment CyberLight was designed for: a university lab, small office, or home setup where nobody is watching a dashboard.

4.3 Advantages

The most obvious advantage is cost. At £37 this is genuinely accessible to a small office or lab that could not justify a commercial IDS. The system is also entirely passive, it reads traffic without injecting anything, which matters both legally and practically. Separating the physical alerting on the Arduino from the network monitoring on the laptop turned out to be a sensible split, since each part only does what it is actually suited for. No specialist dashboard is needed either, which was the whole point from the start.

4.4 Disadvantages and Limitations

- Subnet-limited detection: ARP scanning and passive packet capture detect threats only on the local subnet. Attacks against internet-facing services or from outside the monitored network segment are not visible to CyberLight without integration with a perimeter firewall log.
- Encrypted traffic opacity: The majority of modern network traffic uses TLS encryption. CyberLight can detect attack patterns from metadata (port numbers, packet rates, timing) but cannot inspect payload content of encrypted connections. Sophisticated attackers using encrypted channels over standard ports may evade detection.
- MAC address spoofing: The rogue device detection relies on MAC addresses, which can be spoofed by a knowledgeable attacker. An attacker who clones a whitelisted device's MAC address would not trigger a new-device alert.
- Windows dependency: The brute-force detection mechanism relies on the Windows Security Event Log. If the gateway machine is not a Windows system, this detection method requires replacement with an OS-appropriate equivalent.
- Single point of failure: The Windows gateway runs the detection, ML inference, and MQTT publishing. If the laptop is closed, sleeps, or loses battery, monitoring ceases. A dedicated always-on device (e.g., Raspberry Pi 4) would address this limitation in a production deployment.

4.5 Future Improvements

1. The most practical improvement would be replacing the Windows laptop with a Raspberry Pi 4. The current setup stops monitoring whenever the laptop closes or runs out of battery, which defeats the purpose of an always-on alert system.

2. TLS traffic fingerprinting using JA3: JA3 fingerprinting identifies TLS clients by analyzing the ClientHello packet structure before encryption begins, allowing identification of known-malicious clients without payload decryption. This is used in commercial IDS products and would extend CyberLight's detection capability to encrypted attack traffic.

3. Full ML training: the classifier is built and documented but could not be trained properly in the time available. Given labelled data from controlled test runs, it should be evaluated with precision, recall, and F1, and a feature importance plot would show which traffic features actually matter.

4. Dynamic device whitelist: Integrating DHCP lease log parsing would allow the system to automatically update the known-device whitelist as authorized devices join and leave the network, reducing false positives from legitimate new connections.

5. Mobile push notifications: Routing MQTT events through a Node-RED automation server would enable Telegram or email push notifications to the operator's phone, providing alerting when the operator is not in the room to observe the physical indicators.

6. Multi-sensor data fusion for ML: Adding the DHT11 temperature reading as an environmental feature — a sustained temperature rise in an empty room can indicate human presence — would improve the ML classifier's ability to correlate physical and network anomalies.

4.6 Conclusion

The main goal of this project was to build something that makes network attacks noticeable to people who would not normally catch them. Based on the testing done, CyberLight does that. Running a port scan against the local network triggered an alert in under 10 seconds, sent the event to HiveMQ Cloud, and would drive the physical outputs on the Arduino once the hardware is fully assembled. The core pipeline works.

There are obvious limitations. The system only monitors one subnet, MAC addresses can be spoofed, and the Random Forest classifier is documented but not trained due to time constraints on collecting labelled traffic data. These are worth being honest about because they point to exactly what a better version of this would fix.

For the cost involved, the results are reasonable. A £37 kit will not replace a SIEM platform, but for a small office or lab where no monitoring exists at all, having a red light and a buzzer go off during a port scan is better than nothing.

.

ACKNOWLEDGMENT

Thanks to the M30226 teaching team for support throughout the module. The Elegoo kit was provided by the university for this coursework.

REFERENCES

[1] IBM Security, "Cost of a Data Breach Report 2023," IBM Corporation, 2023. [Online]. Available: https://www.ibm.com/reports/data-breach

[2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," IEEE Communications Surveys and Tutorials, vol. 17, no. 4, pp. 2347-2376, 2015.

[3] Elegoo, "UNO R3 Most Complete Starter Kit — Component List (63 Parts)," Elegoo Official, 2024. [Online]. Available: https://www.elegoo.com/en-gb/products/elegoo-uno-most-complete-starter-kit

[4] A. Shumailov, J. Ruan, R. Anderson, and R. Mullins, "Sponge Examples: Energy-Latency Attacks on Neural Networks," in Proc. IEEE European Symposium on Security and Privacy, 2021.

[5] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Insecure.Com LLC, 2009. [Online]. Available: https://nmap.org/book/

[6] A. Antonakakis et al., "Understanding the Mirai Botnet," in Proc. 26th USENIX Security Symposium, Vancouver, BC, Canada, Aug. 2017, pp. 1093-1110.

[7] Microsoft, "wevtutil," Microsoft Documentation, 2023. [Online]. Available: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/wevtutil